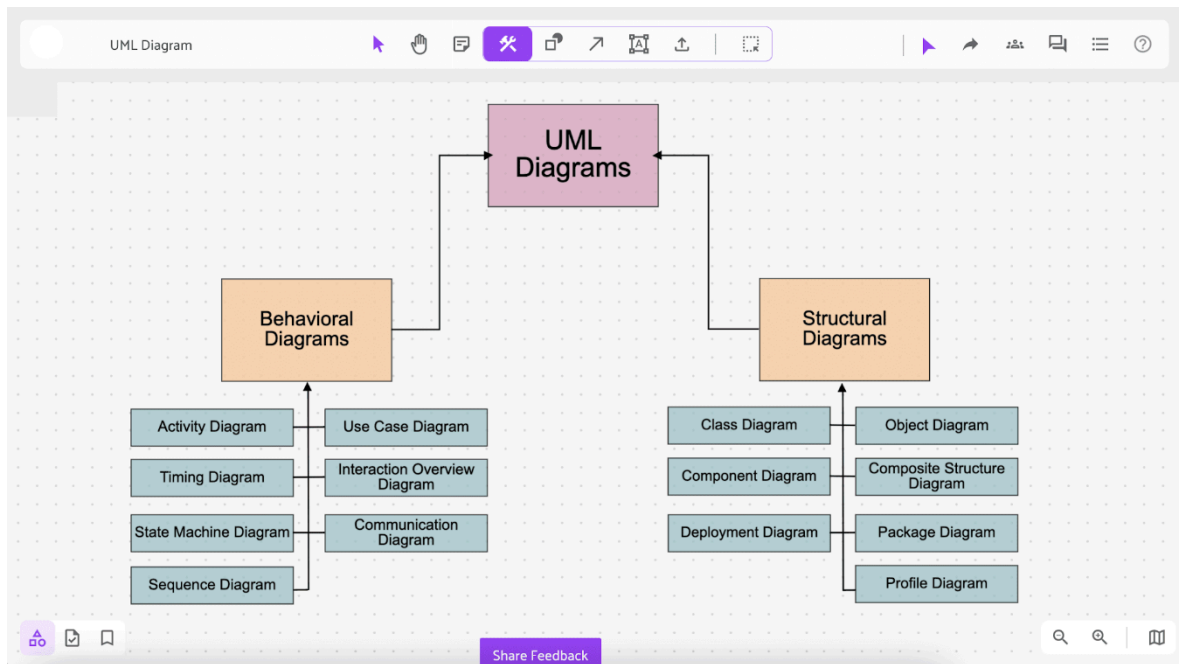
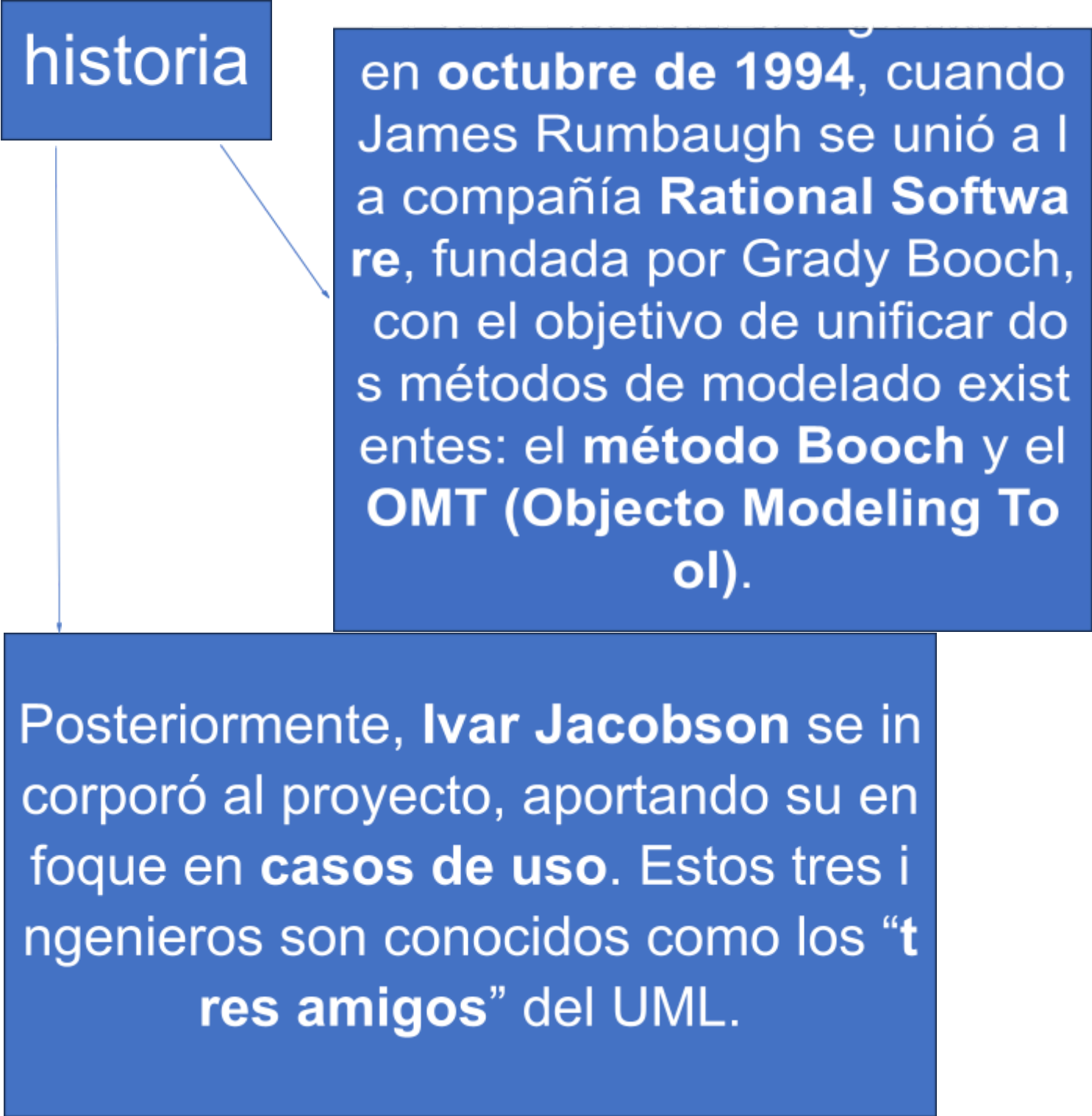


HISTORIA DEL UML



historia



en **octubre de 1994**, cuando James Rumbaugh se unió a la compañía **Rational Software**, fundada por Grady Booch, con el objetivo de unificar dos métodos de modelado existentes: el **método Booch** y el **OMT (Object Modeling Tool)**.

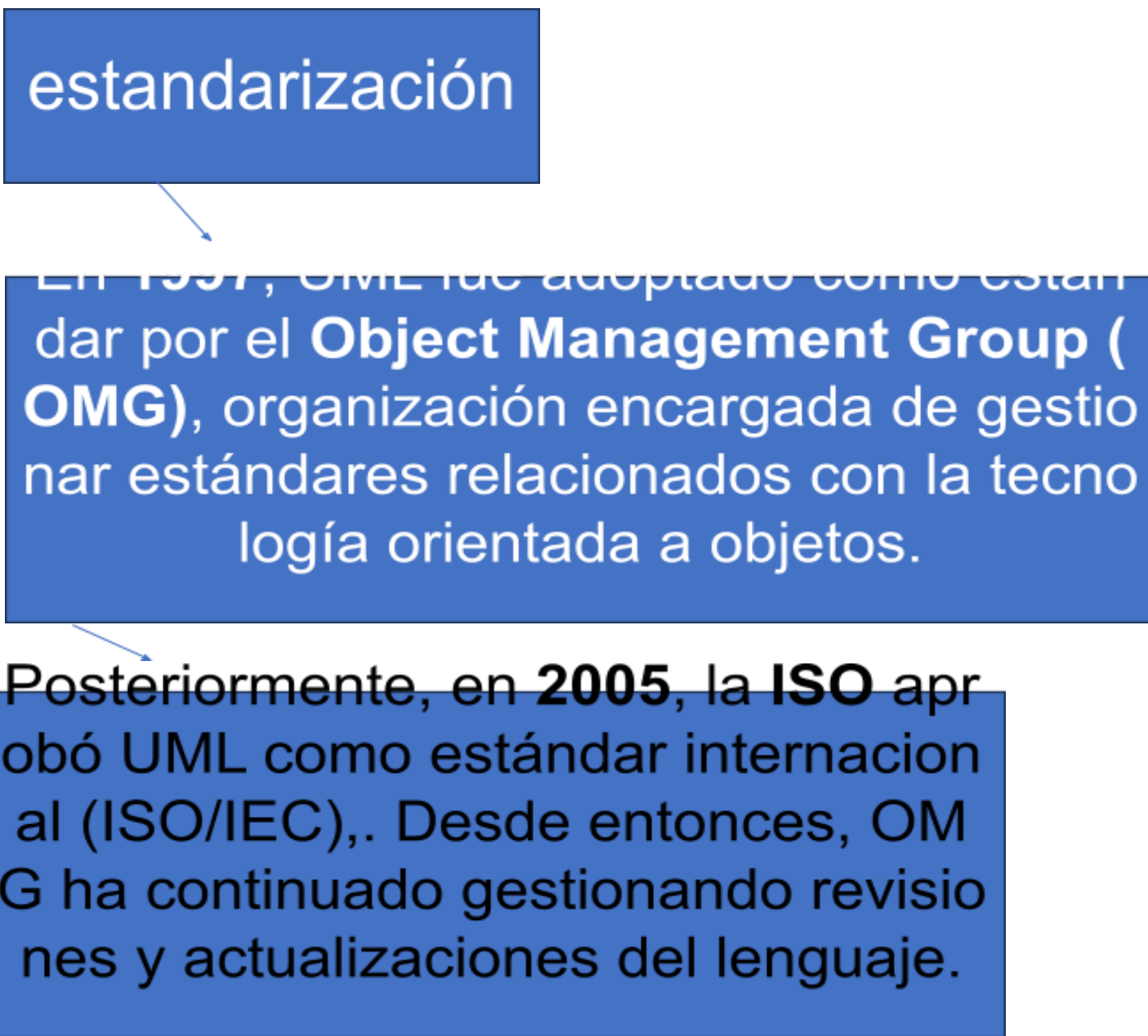
Posteriormente, **Ivar Jacobson** se incorporó al proyecto, aportando su enfoque en **casos de uso**. Estos tres ingenieros son conocidos como los “**tres amigos**” del UML.

origen

El primer borrador del lenguaje apareció en **octubre de 1995**, y se abrió a la colaboración de otras empresas para enriquecer su desarrollo.

La intención era crear un lenguaje que simplificara la **documentación y el diseño de sistemas complejos orientados a objetos**, facilitando la comunicación entre desarrolladores y stakeholders.

estandarización



En 1997, UML fue adoptado como estándar por el **Object Management Group (OMG)**, organización encargada de gestionar estándares relacionados con la tecnología orientada a objetos.

Posteriormente, en **2005**, la **ISO** aprobó UML como estándar internacional (ISO/IEC). Desde entonces, OMG ha continuado gestionando revisiones y actualizaciones del lenguaje.

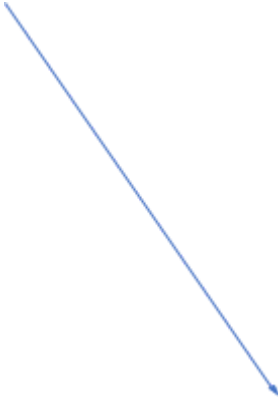
UML 1.0 (1997): Primera versión oficial, con diagramas básicos para modelar sistemas orientados a objetos.

UML 1.x: Varias revisiones hasta la versión 1.4, con mejoras en notación y semántica.

UML 2.x: Introducción de diagramas más complejos, simplificación de notaciones y mejor integración con metodologías ágiles. La versión **2.5** se publicó en **2015**, y la última versión conocida es **2.5.1 (2017)**.

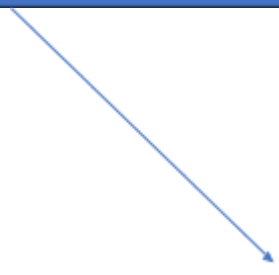
Cada versión buscó **mejorar la funcionalidad y adaptabilidad** del lenguaje, incorporando nuevos tipos de diagramas y refinando la notación para facilitar la comprensión de sistemas complejos.

Importancia del uml



visualizar, modificar, construir y documentar sistemas de software, y su notación gráfica permite representar diferentes aspectos de un sistema, desde **casos de uso y diagramas de clases** hasta **diagramas de despliegue y secuencia**. Además, su flexibilidad permite modelar no solo software, sino también **procesos de negocio, estructuras organizacionales y hardware**.

Resumen

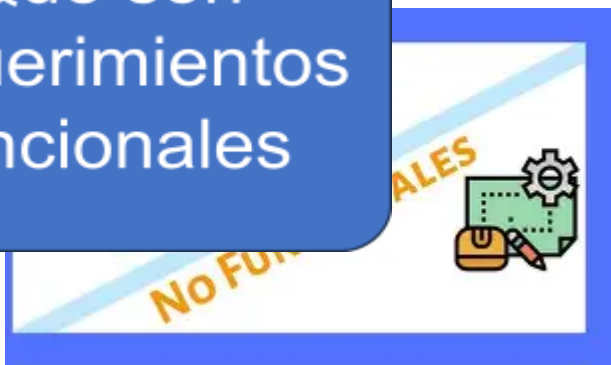


UML surgió como una respuesta a la necesidad de un lenguaje unificado para modelar sistemas complejos, consolidándose como un estándar ampliamente aceptado en la industria del software y evolucionando continuamente para adaptarse a nuevas metodologías y tecnologías.

REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES



Que son
requerimientos
funcionales



Un **requerimiento funcional** es una descripción detallada de lo que un sistema, software o producto debe hacer, incluyendo las acciones, procesos y respuestas esperadas ante distintas entradas o eventos. Estos requisitos permiten traducir las necesidades del usuario y los objetivos del sistema en funciones específicas.

características

```
graph TD; A[características] --> B[Qué debe hacer el sistema]; A --> C[Cómo debe comportarse el sistema];
```

Qué debe hacer el sistema
: Define las tareas y operaciones que el sistema debe ejecutar para cumplir su propósito.

Cómo debe comportarse el sistema: Especifica las respuestas esperadas ante diferentes situaciones o entradas.

Quién interactúa con el sistema: Identifica los actores o usuarios, ya sean personas o sistemas externos.

Condiciones de operación. Incluye manejo de errores, validaciones y procesos específicos.

```
graph TD; A[Condiciones de operación] --> B[Integraciones necesarias]; B --> C[Importancia]; C --> D[Permiten verificar y validar]; C --> E[Ayudan a evitar malentendidos];
```

Integraciones necesarias. Describe cómo el sistema interactúa con otros sistemas, bases de datos o servicios externos.

Importancia

Sirven como base para el **diseño y desarrollo** del sistema.

Permiten verificar y validar que el sistema cumple con las expectativas de los usuarios.

Ayudan a evitar malentendidos y errores de diseño que podrían generar soluciones incompletas o ineficaces.

Facilitan la documentación clara y rastreable, asegurando que cada función esté alineada con un objetivo de negocio.

ejemplos

```
graph TD; A[ejemplos] --> B[Autenticación de usuario]; A --> C[Gestión de inventario]; A --> D[Soporte multilingüe]; A --> E[Generación de reportes]; A --> F[Procesamiento de pagos];
```

Autenticación de usuario: Registro e inicio de sesión con nombre de usuario y contraseña.

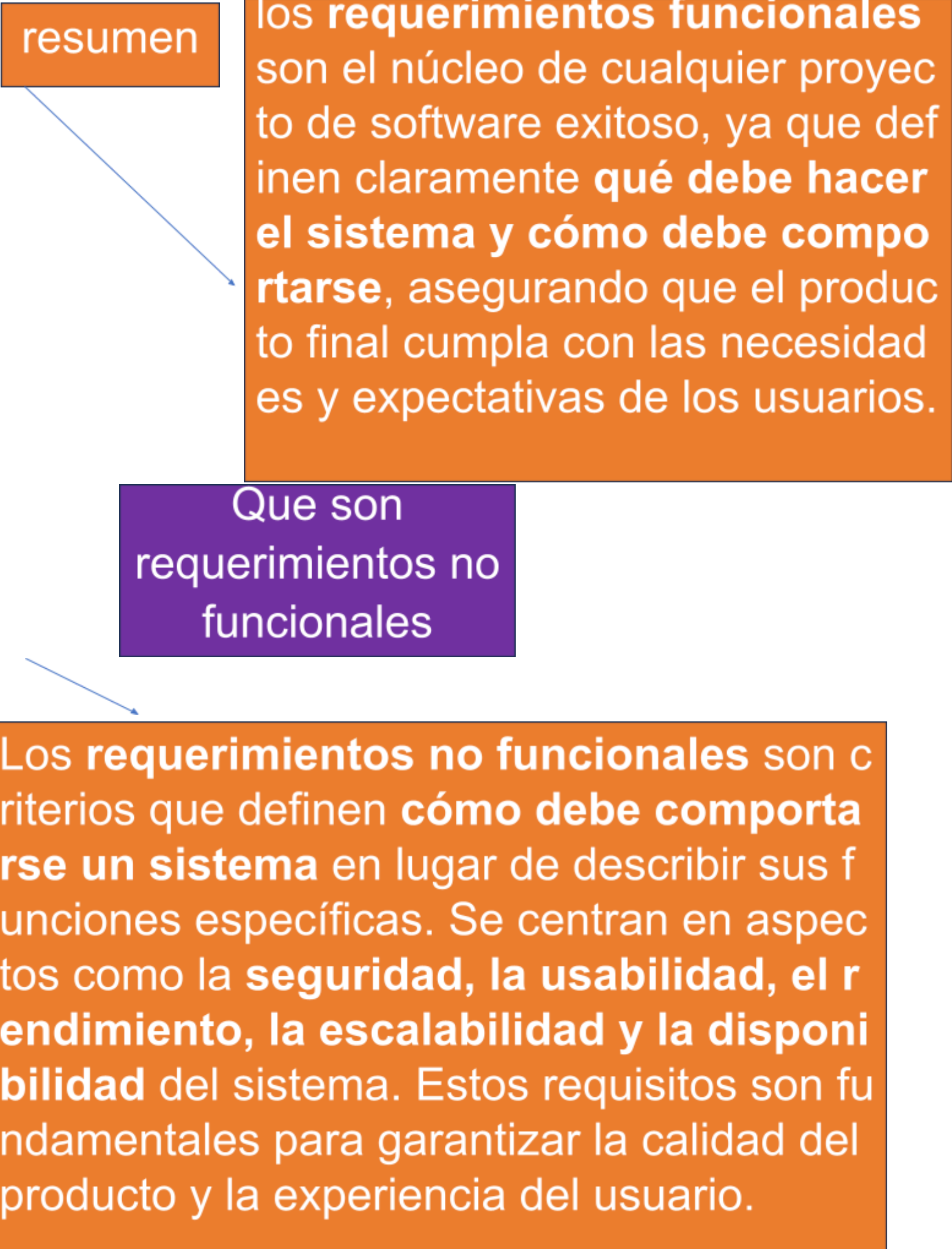
Gestión de inventario: Agregar, editar, eliminar y buscar productos en la base de datos.

Soporte multilingüe: Interfaces en diferentes idiomas para usuarios globales.

Generación de reportes: Creación automática de informes de ventas, gastos o indicadores de desempeño.

Procesamiento de pagos: Integración de un módulo seguro para transacciones con tarjetas.

resumen



```
graph TD; A[resumen] --> B[los requerimientos funcionales son el núcleo de cualquier proyecto de software exitoso, ya que definen claramente qué debe hacer el sistema y cómo debe comportarse, asegurando que el producto final cumpla con las necesidades y expectativas de los usuarios.]; B --> C[Que son requerimientos no funcionales]; C --> D[Los requerimientos no funcionales son criterios que definen cómo debe comportarse un sistema en lugar de describir sus funciones específicas. Se centran en aspectos como la seguridad, la usabilidad, el rendimiento, la escalabilidad y la disponibilidad del sistema. Estos requisitos son fundamentales para garantizar la calidad del producto y la experiencia del usuario.];
```

los **requerimientos funcionales** son el núcleo de cualquier proyecto de software exitoso, ya que definen claramente **qué debe hacer el sistema y cómo debe comportarse**, asegurando que el producto final cumpla con las necesidades y expectativas de los usuarios.

Que son
requerimientos no
funcionales

Los **requerimientos no funcionales** son criterios que definen **cómo debe comportarse un sistema** en lugar de describir sus funciones específicas. Se centran en aspectos como la **seguridad, la usabilidad, el rendimiento, la escalabilidad y la disponibilidad** del sistema. Estos requisitos son fundamentales para garantizar la calidad del producto y la experiencia del usuario.

características

```
graph TD; A[características] --> B[Los requerimientos no funcionales son características y criterios que describen cómo debe ser el rendimiento, la seguridad.]; A --> B; B --> C[eficiente y cumpla con las expectativas y necesidades del usuario. Al definir adecuadamente estos requisitos.]; C --> D[la usabilidad y otros aspectos de un sistema o software más allá de su funcionalidad básica. Estos requisitos son cruciales para garantizar que el software sea eficaz.]; D --> E[Los equipos de desarrollo pueden anticiparse a problemas potenciales y asegurar que el software no solo cumpla con las especificaciones funcionales, sino que también sea eficiente y confiable en su uso diario.];
```

Los requerimientos no funcionales son características y criterios que describen cómo debe ser el rendimiento, la seguridad.

eficiente y cumpla con las expectativas y necesidades del usuario. Al definir adecuadamente estos requisitos.

la usabilidad y otros aspectos de un sistema o software más allá de su funcionalidad básica.

Estos requisitos son cruciales para garantizar que el software sea eficaz.

Los equipos de desarrollo pueden anticiparse a problemas potenciales y asegurar que el software no solo cumpla con las especificaciones funcionales, sino que también sea eficiente y confiable en su uso diario.

elementos

```
graph TD; A[elementos] --> B[Rendimiento]; A --> C[Usabilidad]; C --> D[Seguridad];
```

Rendimiento: Define tiempos de respuesta máximos, capacidad de procesamiento simultáneo y volúmenes de datos que el sistema debe soportar.

Usabilidad: Establece criterios de facilidad de uso, accesibilidad, consistencia visual y tiempo de aprendizaje aceptable.

Seguridad: Incluye autenticación, autorización, cifrado de datos, auditoría y protección frente a accesos no autorizados.

Confiabilidad: Describe la estabilidad del sistema, tolerancia a fallos y probabilidad de errores durante la operación.



Disponibilidad: Indica el porcentaje de tiempo que el sistema está disponible para los usuarios.



Estos elementos son fundamentales para garantizar que el software sea robusto, confiable y agradable de usar en la práctica.

importancia



Utilizar requisitos no funcionales es una parte importante al momento de desarrollar un sistema, **hará que tu software o aplicación sea eficaz, eficiente y cumpla con las expectativas y necesidades del usuario.** No obstante, debes entender que estos trabajan de la mano con los requisitos funcionales.

conclusión



Los requerimientos no funcionales son esenciales para el éxito de un proyecto de software. Definen cómo debe comportarse un sistema más allá de lo que hace, abarcando aspectos como el rendimiento, la seguridad, la usabilidad y la escalabilidad.

